



Bring Your Own Agent

LangGraph in production on Foundry Agent Service

Andrey Vykhodtsev
mailto: andy.v@ai-bistro.com
Linkedin/github: @vykhand

NT konferenca 2026 · Portorož · 21–23 September 2026

It works on my laptop



Security: which identity it uses to reach your systems

Operations: where the logs are when it answers wrong

Finance: what it costs, and why it ran all night

The example: an agenda for this conference



- „Zanima me AI in Azure, v torek, nič pred deveto.“ (AI and Azure, Tuesday, nothing before nine)
- It searches the programme, fits the times, leaves a coffee break
- Three tools behind it; it answers in Slovene

LangChain and LangGraph



LangChain: interfaces to models and tools · LangGraph: runs the agent as a graph

Who does what

Your agent

graph.py · three tools · a prompt

LangGraph

runs the graph: state, nodes, edges, the loop

LangChain

one interface to models and tools

Model

gpt-5.4-mini on Foundry · gpt-oss:20b on the laptop

Hosting

Foundry hosted
agent (cloud)

hosting.serve
(laptop, :8088)

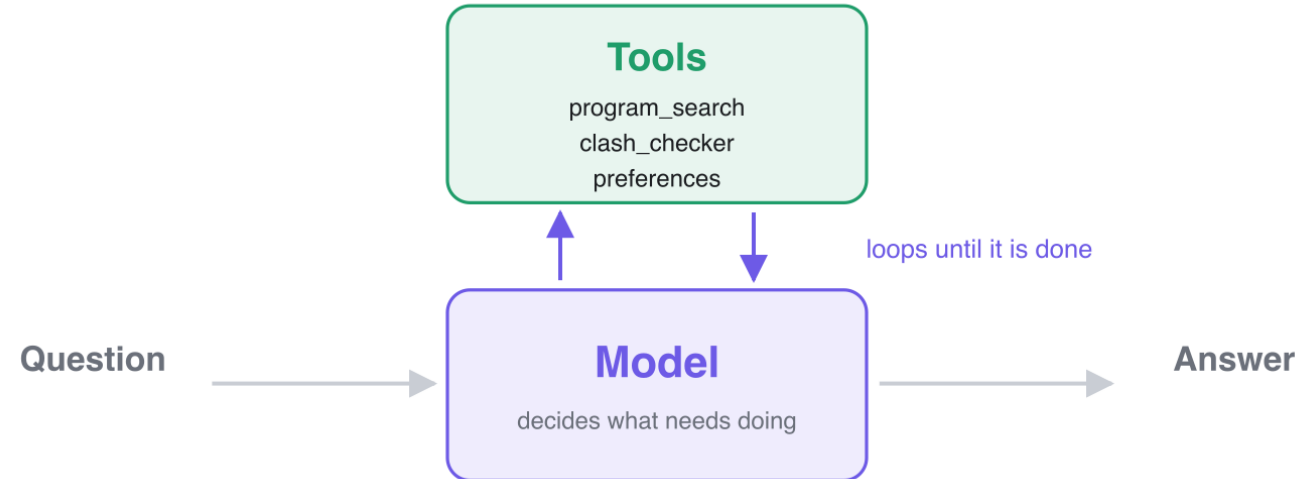
Only the hosting layer imports Azure code.

An agent is a model, tools and a loop



The model decides; plain code runs the tools · The loop stops when no tool is called

What an agent is



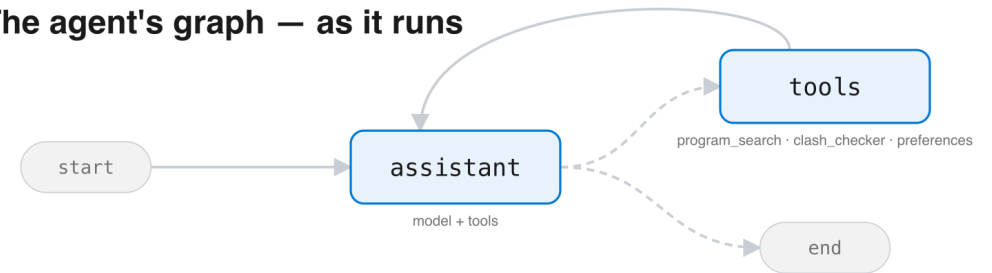
An agent is not a model. It is a model allowed to do something.

LangGraph: state, nodes, edges



- State: one typed object that every step reads and updates
- Nodes: plain Python functions — the model call, the tool runner
- Edges: fixed or conditional; the conditional edge makes the loop
- Checkpointer: saves the state per thread, for memory and resuming

The agent's graph — as it runs



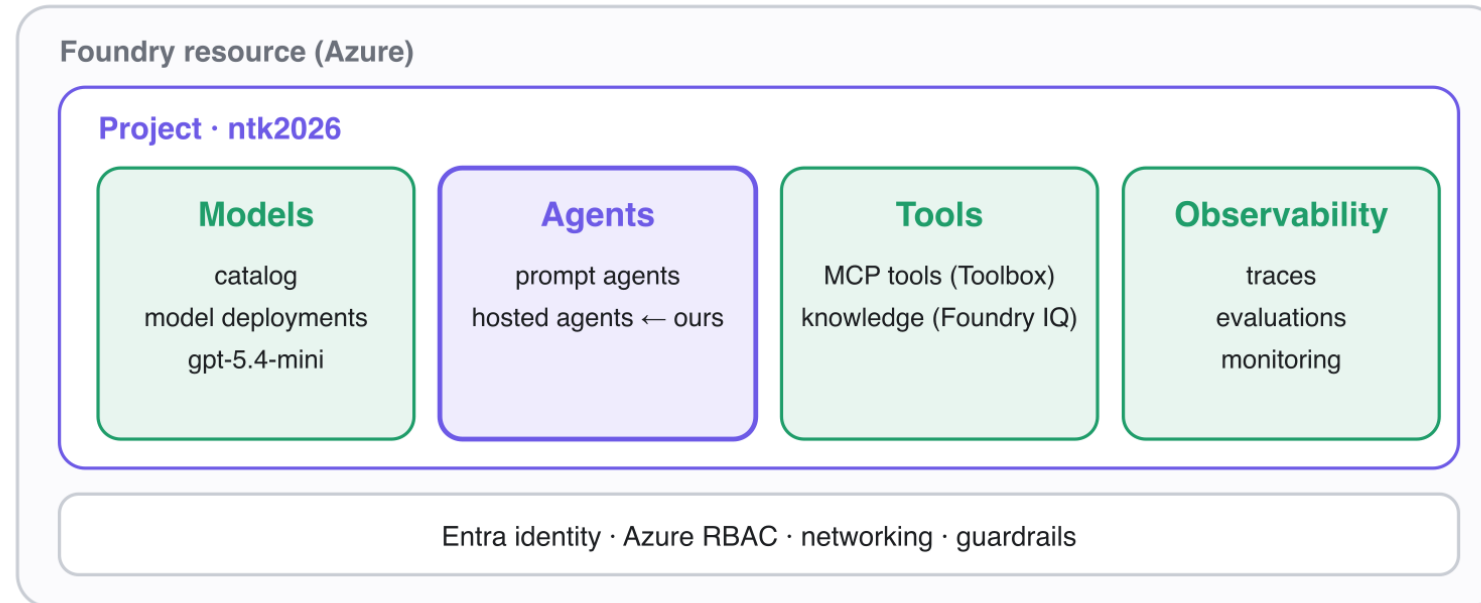
The model decides; if it needs a tool it calls one and comes back. No Azure code.

Microsoft Foundry



Azure's platform for AI apps and agents · Projects hold models, agents, tools and traces

Microsoft Foundry in one picture



Our agent is one hosted agent in one project.

Foundry Agent Service: two kinds of agent

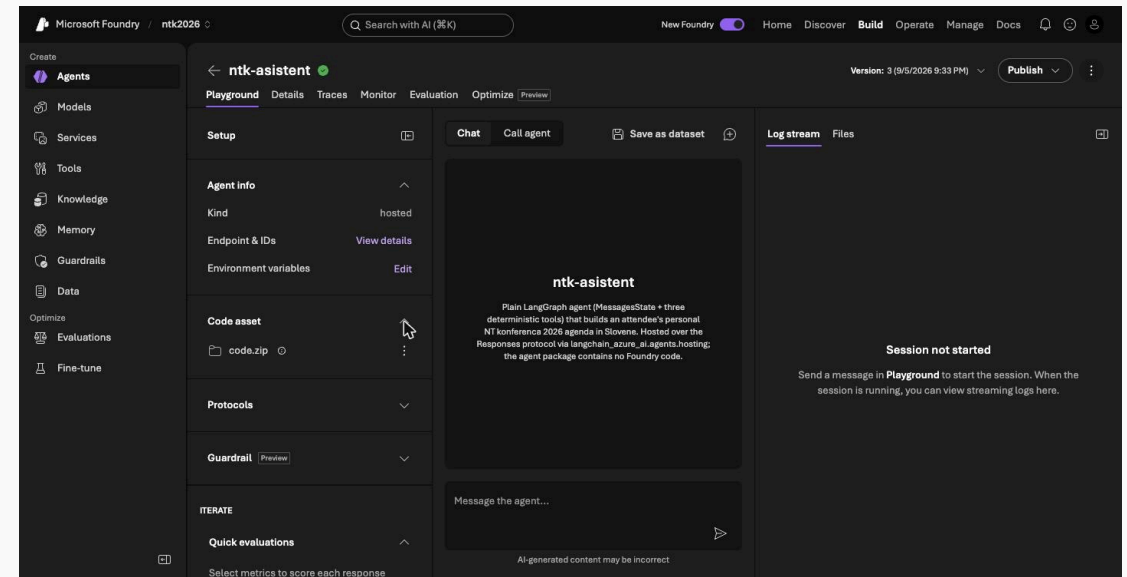


Kind	You provide	Fits
Prompt agent	instructions, a model, tools — configuration, no code	assistants built in the portal or SDK
Hosted agent	your own code: LangGraph, Agent Framework, OpenAI Agents SDK, ...	agents you already wrote

Hosted agents: how your code runs



- azd deploy uploads the code; Foundry builds it and runs it
- An adapter serves the graph over the Responses API, on port 8088
- Each session runs in its own VM-isolated sandbox
- Each agent gets its own Entra identity and endpoint

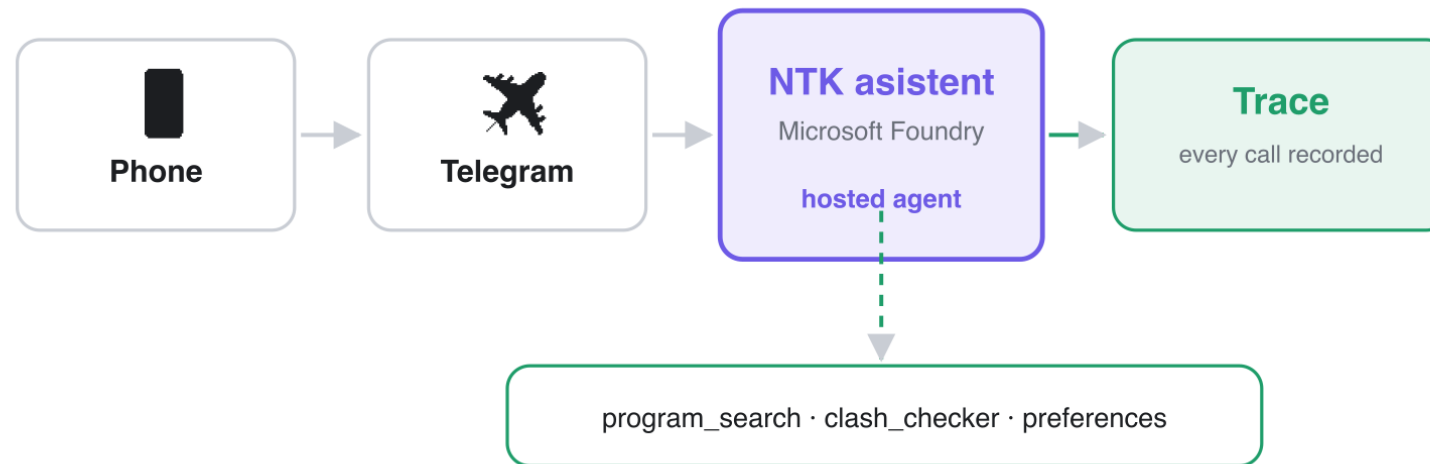


The demo path

Graph → deploy → traces → evaluations · Every step leaves a trace



The demo: from a phone to a trace



The bot is only a courier. The agent in Azure does the work.

Three tools: what goes in, what comes out



Tool	Goes in	Comes out
`preferences`	an attendee's name, or nothing	JSON profile: interests, tracks, earliest start, latest end
`program_search`	keywords, track, day, room	JSON list of talks: id, title, room, start, end
`clash_checker`	the ids of the chosen talks	JSON: overlapping pairs, unknown ids, ok or not

A tool is a function with a docstring



src/agent/tools.py — one of the three tools, complete

```
@tool
def preferences(attendee: str | None = None) -> str:
    """Read an attendee's profile: interests, preferred tracks, days and constraints.

    Args:
        attendee: Attendee id or first name. Omit to use the attendee this session is configured for.

    Returns:
        JSON profile with interests, tracks, days and constraints (earliest_start, latest_end,
        min_break_minutes, lunch_break, preferred_language) plus a short note in Slovene.
    """
    try:
        return _json(get_persona(attendee))
    except KeyError as exc:
        return _json({"error": str(exc)})
```

One question, step by step



Step	Goes in	Comes out
Model	the messages so far and the three tool schemas	tool calls — names and JSON arguments — or the answer
Tool node	the tool calls	one tool message per call, with the function's JSON result
Edge	the updated state	back to the model after tools; the end when it answers



DEMO 1 • The agent is a graph

The agent's three tools – agent.tools.default_tools()

Search...				Columns	Explore	Export
tool	what it does	arguments				
program_search	Search the NT konferenca 2026 program.	query, track, day, room, limit				
clash_checker	Check a candidate agenda for overlapping talks.	talk_ids				
preferences	Read an attendee's profile: interests, preferred track...	attendee				
3 rows, 3 columns				No selection		

src/agent/graph.py – the body of build_graph

```
1 def build_graph(  
2     model: BaseChatModel,  
3     *,  
4     system_prompt: str | None = None,  
5     tools: Sequence[BaseTool] | None = None,  
6     enforce_constraints: bool = False,  
7 ) -> CompiledStateGraph:  
8     """Compile the agent graph around any LangChain chat model that supports tool calling."""  
9     toolset = list(tools) if tools is not None else default_tools()  
10    prompt = system_prompt if system_prompt is not None else load_system_prompt()  
11    model_with_tools = model.bind_tools(toolset)  
12  
13    async def assistant(state: MessagesState) -> dict:  
14        response = await model_with_tools.ainvoke([SystemMessage(content=prompt), *state["messages"]])  
15        return {"messages": [response]}  
16  
17    def guard_node(state: MessagesState) -> dict:  
18        """Score the answer the model just wrote; hand the violations back, or pass.  
19  
20        Every decision is logged, including the passes: an approval and a guard that never ran  
21        look identical from the outside, which is how the residual failures in S-08b went  
22        unexplained for a day. ``attendee_source`` is the field that earns its keep.  
23  
24        ``RemoveMessage`` drops the rejected answer from state. and **that does not un-send it**
```

Flip the switch. The graph below is redrawn from the compiled code 

☐ guard (enforce_constraints)

Guard OFF – two nodes: **assistant** → **tools**

start

The hosting wrapper: 14 lines



src/hosting/serve.py — the entire hosting wrapper

```
"""The entire Foundry hosting wrapper. Everything above this line is plain LangGraph."""
import os

from langchain_azure_ai.agents.hosting import ResponsesHostServer

from hosting.app import graph

def main() -> None:
    ResponsesHostServer(graph).run(port=int(os.environ.get("PORT", "8088")))

if __name__ == "__main__":
    main()
```



DEMO 2 · Publishing with one command

src/hosting/serve.py — the entire Foundry integration. 14 lines.

```
1 """The entire Foundry hosting wrapper. Everything above this line is plain LangGraph."""
2 import os
3
4 from langchain_azure_ai.agents.hosting import ResponsesHostServer
5
6 from hosting.app import graph
7
8
9 def main() -> None:
10     ResponsesHostServer(graph).run(port=int(os.environ.get("PORT", "8088")))
11
12
13 if __name__ == "__main__":
14     main()
```

langgraph.json — tells the platform where the graph is.

```
1 {
2   "graphs": {
3     "agent": "./src/hosting/app.py:graph"
4   }
5 }
```

azure.yaml, service ntk-asistent — entry point and environment variables. Every change here creates a new version.

```
1 ntk-asistent:
2   host: azure.ai.agent
3   kind: hosted
4   name: ntk-asistent
5   displayName: NTK asistent
6   description: >-
7     Plain LangGraph agent (MessagesState + three deterministic tools) that builds an attendee's
8     personal NT konferenca 2026 agenda in Slovene. Hosted over the Responses protocol via
9     langchain_azure_ai.agents.hosting; the agent package contains no Foundry code.
10  metadata:
11    tags:
12      - AI Agent Hosting
13      - Responses Protocol
14      - LangGraph
15      - LangChain
16      - Python
17      - NTK 2026
18    # The whole repo is the deployable unit; .agentignore trims the ZIP to src/, fixtures/, prompts/.
19  project: .
20  language: python
21  codeConfiguration:
22    runtime: python_3_13
23    entryPoint: '-m langchain azure ai.agents.hosting.run --protocol responses'
```

In the portal: the playground

Chat with the deployed agent, no client code · Logs, code asset and settings on one page



The screenshot shows the Microsoft Foundry portal interface for an agent named 'ntk-asistent'. The left sidebar contains navigation options: Agents, Models, Services, Tools, Knowledge, Memory, Guardrails, Data, Optimizations, Evaluations, and Fine-tune. The main area is divided into three panels. The left panel shows agent details: Kind (hosted), Endpoint & IDs (with a 'View details' link), Environment variables (with an 'Edit' link), Code asset, Protocols, and Guardrail (with a 'Preview' button). The middle panel has tabs for 'Chat' and 'Call agent', and a 'Save as dataset' button. It displays the agent's description: 'Plain LangGraph agent (MessagesState + three deterministic tools) that builds an attendee's personal NT konferenca 2026 agenda in Slovene. Hosted over the Responses protocol via langchain_azure_ai.agents.hosting; the agent package contains no Foundry code.' Below this is a text input field 'Message the agent...' and a send button. The right panel has tabs for 'Log stream' and 'Files', and shows a 'Session not started' message with instructions to send a message in the playground to start the session and view streaming logs.

In the portal: identity and endpoint

Its own Entra identity; roles go to the agent · One endpoint: Responses API, Azure RBAC



A screenshot of the Microsoft Foundry portal interface. The top navigation bar includes 'Microsoft Foundry / ntk2026', a search bar, and links for 'New Foundry', 'Home', 'Discover', 'Build', 'Operate', 'Manage', 'Docs', and user icons. A left sidebar lists 'Create' (Agents, Models, Services, Tools, Knowledge, Memory, Guardrails, Data) and 'Optimize' (Evaluations, Fine-tune). The main content area is for the 'ntk-asistent' agent, with tabs for 'Playground', 'Details' (selected), 'Traces', 'Monitor', 'Evaluation', 'Optimize', and 'Preview'. The 'Details' tab shows 'Agent configuration' with 'Status' (Running) and 'Active version' (Latest (Version 3)). The 'Identity & access' section lists 'Entra agent identity' and 'Entra agent blueprint' with their respective IDs. The 'Endpoints' section shows the 'Responses protocol' as 'Active' with a note 'People with Azure RBAC permissions can access' and an endpoint URL. The 'A2A protocol' has a 'Set up' button. The 'Channels' section at the bottom lists 'Teams & Microsoft 365 Copilot', 'Preview web app', and 'Routines'.

In the portal: versions

Every deploy is a new, immutable version · One version is active; going back is one call



A screenshot of the Microsoft Foundry portal interface. The top navigation bar includes 'Microsoft Foundry / ntk2026', a search bar, and tabs for 'New Foundry', 'Home', 'Discover', 'Build', 'Operate', 'Manage', 'Docs', and user icons. A left sidebar lists 'Create' (Agents, Models, Services, Tools, Knowledge, Memory, Guardrails, Data) and 'Optimize' (Evaluations, Fine-tune). The main content area is for the 'ntk-asistent' agent, with tabs for 'Playground', 'Details' (selected), 'Traces', 'Monitor', 'Evaluation', 'Optimize', and 'Preview'. The 'Details' tab shows 'Agent configuration' with 'Status' (Running), 'Active version', and 'Latest (Version 3)'. The 'Identity & access' section lists 'Entra agent identity' and 'Entra agent blueprint'. The 'Endpoints' section shows 'Responses protocol' (Active) and 'A2A protocol' (Set up). The 'Channels' section lists 'Teams & Microsoft 365 Copilot', 'Preview web app', and 'Routines'. A version dropdown menu is open, showing 'Version 3' (Active), 'Version 2', and 'Version 1', along with options to 'Compare versions', 'Show all version history', and 'Delete current version'.

In the portal: traces



One row per call: status, duration, tokens, cost · Trace, conversation and session views

The screenshot shows the Microsoft Foundry portal interface. The left sidebar contains navigation options: Agents, Models, Services, Tools, Knowledge, Memory, Guardrails, Data, Optimizations, Evaluations, and Fine-tune. The main content area is titled 'ntk-asistent' and shows the 'Traces' view. The 'Trace view' tab is selected, displaying a table of agent invocation spans. The table has columns for Trace ID, Status, Created at, Duration (s), Tokens (In), Tokens (Out), Estimated cost (€), Evaluation, Annotation, and Agent version. A single trace is listed with ID 'ff35f39fd6a...', Status 'Completed', Created at '9/6/26, 2:12:35 AM', Duration '7.053', Tokens (In) '0', Tokens (Out) '0', Estimated cost '-' , Evaluation '--', and Agent version '3'. The interface also includes search filters, a date range selector, and a 'Create dataset' button.

Trace ID	Status	Created at	Duration (s)	Tokens (In)	Tokens (Out)	Estimated cost (€)	Evaluation	Annotation	Agent version
ff35f39fd6a...	Completed	9/6/26, 2:12:35 AM	7.053	0	0	-	--	Annotate	3

In the portal: one call, step by step

The loop from Demo 1: model, tool, model, tool, answer · Nothing instrumented by hand

A screenshot of the Microsoft Foundry portal interface. The main window displays a detailed trace for an agent named 'invoke_agent'. The trace is a hierarchical tree of events, including HTTP requests, chat messages, and tool executions, each with a duration bar. On the right, a 'Metadata' panel shows a JSON object with agent details. The interface includes a sidebar with navigation options like 'Agents', 'Models', 'Tools', and 'Knowledge', and a top navigation bar with 'Home', 'Discover', 'Build', 'Operate', 'Manage', and 'Docs'.

invoke_agent ID: ff35f39fd6a404ff4a6abc321925bb7f

Trajectories User view Graph view

Find in trace All Duration

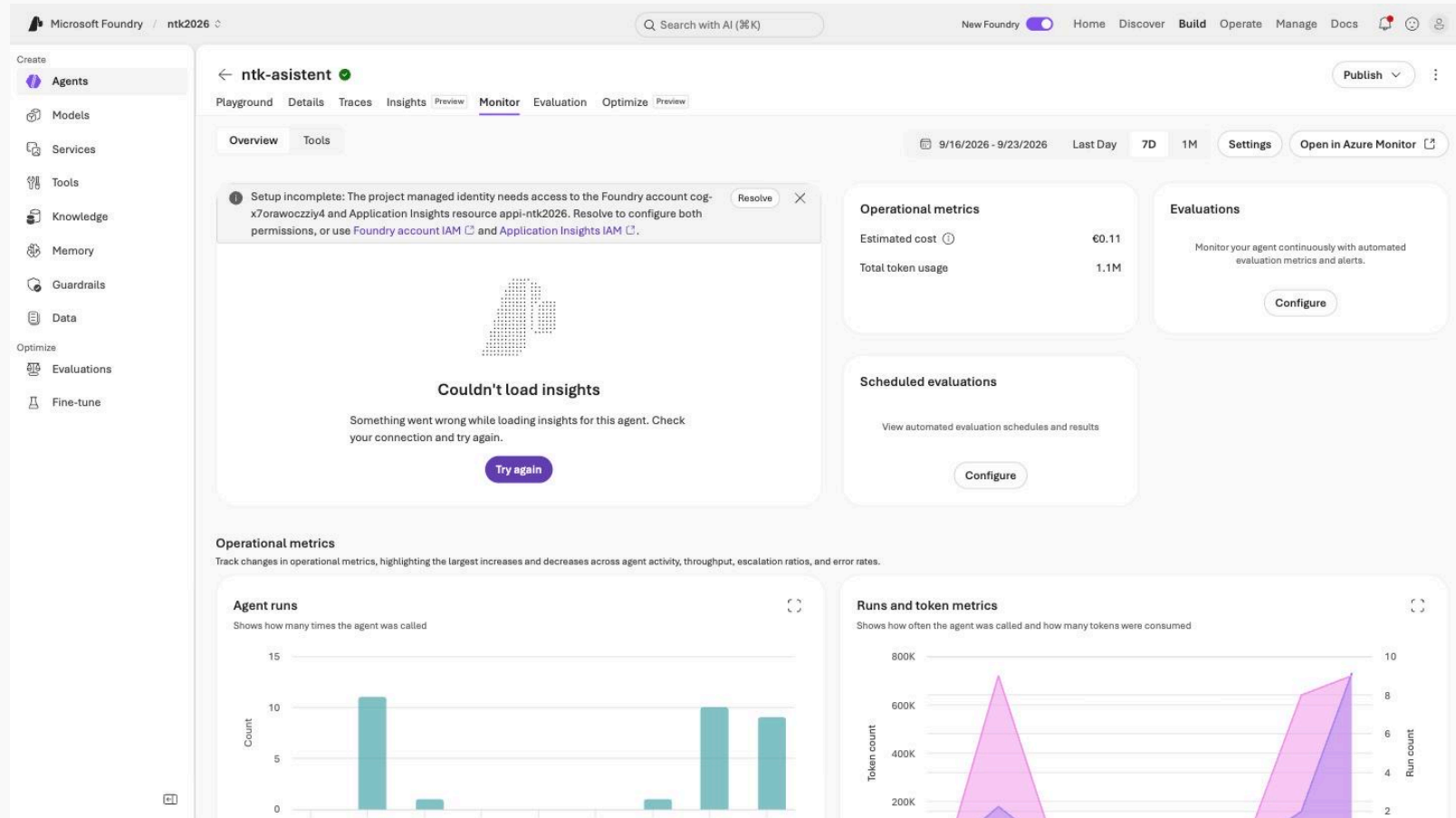
invoke_agent ID: 63e4f2d21abcd15b 7.05s

Metadata Annotations

```
1 {
2   "name": "invoke_agent",
3   "context": {
4     "trace_id":
5       "ff35f39fd6a404ff4a6abc321925bb7f",
6     "span_id": "63e4f2d21abcd15b"
7   },
8   "kind": "default",
9   "start_time": "2026-09-
10    06T00:12:35.107656Z",
11   "end_time": "2026-09-06T00:12:42.160Z",
12   "status": {
13     "status_code": "Ok"
14   },
15   "attributes": {
16     "span_type": "default",
17     "_MS.ResourceAttributeId": "9fa795f5-
18       b34e-44e4-98dc-d62f38d08a5b",
19     "gen_ai.operation.name": "invoke_agent",
20     "gen_ai.agent.type": "AzureAIFoundry",
21     "microsoft.foundry.agent.type":
22       "hosted",
23     "microsoft.channel.name":
24       "agent_service",
25     "agent.name": "ntk-asistent",
26     "gen_ai.agent.name": "ntk-asistent",
27     "gen_ai.agent.id": "4394417e-cae7-40a3-
28       8ea7-da8e7e8ac81b",
29     "agent.version": 3,
30     "gen_ai.agent.version": 3,
31   }
32 }
```

In the portal: monitor

Runs, tokens and estimated cost over time · Scheduled evaluations and alerts start here



In the portal: optimize

Tries other instructions, tool texts, models · Your evaluators rank them; you pick one

A screenshot of the Microsoft Foundry web portal. The top navigation bar includes 'Microsoft Foundry / ntk2026', a search bar, and links for 'New Foundry', 'Home', 'Discover', 'Build', 'Operate', 'Manage', and 'Docs'. A left sidebar lists categories: 'Create' (Agents, Models, Services, Tools, Knowledge, Memory, Guardrails, Data) and 'Optimize' (Evaluations, Fine-tune). The main content area is for 'ntk-asistent' and shows the 'Optimize' tab. It features a header 'See how much Optimize can improve your agent' with a sub-header 'Example internal benchmark: tax agent'. Below this, a comparison table shows a 'Baseline' score of 75.0 (Original agent configuration) and an 'Optimized candidate' score of 92.5 (+17.5 points, Best configuration found by Optimize). A purple 'Optimize my agent' button is at the bottom, along with links for 'How Optimize works' and 'Read the benchmark story'.

Configuration	Score	Notes
Baseline	75.0	Original agent configuration
Optimized candidate	92.5	+17.5 points Best configuration found by Optimize

What the platform provides



None of this is in the repository · Only traces need setup: App Insights and roles

What you get without writing it



Identity

The agent has its own
Entra account — not
the app's secret.



Isolation

Every conversation
gets its own sandbox.
Nothing leaks.



Versions

Every deploy is a
frozen number. Back
with one command.



Traces

Every call is
recorded: which tools,
how long.

— without a single line of your code —

Write to the agent from your phone

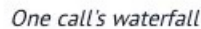


- Telegram: @ntk2026_langgraph_bot
- Write what interests you and when you have to leave
- An answer in ~20 seconds





DEMO 3 · Traces and sandboxes

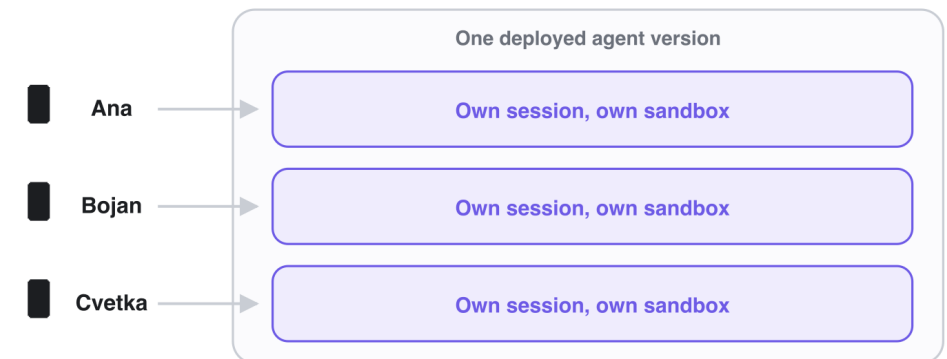
No selection

Conversations and sessions



- Conversation: the message history Foundry stores
- Session: the VM-isolated sandbox that runs the agent
- An idle session releases its compute; the state is kept

Everyone gets their own session



Ana's agenda cannot leak into Bojan's conversation.

Evaluations: the evaluators



Built-in agent evaluators: tool calls, task adherence · Custom ones: code or an LLM judge

Microsoft Foundry / ntk2026

Search with AI (9K)

New Foundry Home Discover Build Operate Manage Docs

Create

- Agents
- Models
- Services
- Tools
- Knowledge
- Memory
- Guardrails
- Data

Optimize

- Evaluations**
- Fine-tune

Evaluations

Evaluations **Evaluator catalog** Red team Preview

An evaluator is an asset that can be used to run an evaluation. You can define an evaluator in SDK and run an evaluation to generate scores of one or more metrics.

Search evaluators Type Category

Create evaluator

Name	Description	Type	Category	Version	Created on	Publisher
☐ NTK · Slovene register (judge)	LLM judge: is the narrative ful...	Custom Prompt	quality	7	9/6/26, 1:10:59 AM	????? ???????
☐ NTK · completeness (judge)	LLM judge: are the requested...	Custom Prompt	quality	7	9/6/26, 1:10:58 AM	????? ???????
☐ NTK · coffee break coverage	1.0 when the agenda leaves ...	Custom Code	quality	7	9/6/26, 1:10:56 AM	????? ???????
☐ NTK · preference violations	1.0 when no talk starts befor...	Custom Code	quality	7	9/6/26, 1:10:54 AM	????? ???????
☐ NTK · clash-free agenda	1.0 when the produced agen...	Custom Code	quality	7	9/6/26, 1:10:51 AM	????? ???????
☐ smoke-core		Custom Rubric	quality, agents	1	9/6/26, 12:56:50 AM	????? ???????
☐ Retrieval-Evaluator	Measures how effectively the...	Built-in	quality	14	9/19/26, 7:13:39 PM	Microsoft
☐ Tool-Selection-Evaluator	Evaluates whether an AI age...	Built-in	agents	13	8/13/26, 9:44:20 PM	Microsoft
☐ Tool-Output-Utilization-Evaluator	Checks if an agent correctly i...	Built-in	agents	11	8/13/26, 9:44:29 PM	Microsoft
☐ Tool-Call-Success-Evaluator	Evaluates whether all tool ca...	Built-in	agents	13	8/13/26, 9:44:22 PM	Microsoft
☐ Tool-Call-Accuracy-Evaluator	Measures whether the agent ...	Built-in	agents	15	8/13/26, 9:44:22 PM	Microsoft
☐ Task-Completion-Evaluator-(Preview)	Evaluates whether an AI age...	Built-in	agents, business	19	8/13/26, 9:44:08 PM	Microsoft
☐ Task-Adherence-Evaluator-(Preview)	Evaluates whether the agent ...	Built-in	agents	16	8/13/26, 9:44:05 PM	Microsoft
☐ Response-Completeness-Evaluator-(Preview)	Assesses whether the respo...	Built-in	quality	10	8/13/26, 9:44:08 PM	Microsoft
☐ Relevance-Evaluator	Assesses how well the respo...	Built-in	quality, agents	12	8/13/26, 9:43:57 PM	Microsoft
☐ Intent-Resolution-Evaluator-(Preview)	Checks whether the model c...	Built-in	agents, quality	9	8/13/26, 9:44:05 PM	Microsoft

Evaluations: one run against the agent



Each test question goes to the live agent · Each evaluator marks each answer pass or fail

Microsoft Foundry / ntk2026

Search with AI (🔍)

New Foundry Home Discover Build Operate Manage Docs

Create

- Agents
- Models
- Services
- Tools
- Knowledge
- Memory
- Guardrails
- Data

Optimize

- Evaluations**

Fine-tune

← ntk-smoke-0012

Make recurring Delete

Run details

Status	Create time	Duration	Created by
Completed	9/6/2026, 2:12:21 AM	55.88s	?????? ????????
Agent ID	See all properties	User logs	Results
ntk-asistent:3	Raw JSON	Download user logs	Download results

Overall metric results

P50 latency	P95 latency	ntk_clash_free	ntk_preferences	ntk_break_coverage
2.9s	15.4s	50% 1/2	50% 1/2	50% 1/2

Detailed metrics result

Index	conversation_id	trace_id	ntk_break_coverage	ntk_clash_free	ntk_preferences	P
0	conv_00229531b5e6f60100DnGbYaO8 BedBD04k01bwIZESkVqalLx	-	Pass: 1	Pass: 1	Pass: 1	3/
1	conv_07a8f050015084bc00fKwNv357 HxGzLXFjqRAT9qJPFbQmIlgk	-	Fail: 0	Fail: 0	Fail: 0	0/

< Prev Next >

Demo 4: evaluating the agent



Recorded click-through: evaluation, run, results · The same pages are live in the portal tab

The screenshot shows the 'Evaluation' tab for the 'ntk-asistent' agent in the Microsoft Foundry portal. The interface includes a left sidebar with navigation options like Agents, Models, Services, Tools, Knowledge, Memory, Guardrails, Data, Evaluations, and Fine-tune. The main content area displays a table of evaluation runs under the 'Automatic Evaluation' tab. The table has columns for Name, Status of last run, Last run, Runs, Frequency, Created by, and Created on. The first run, 'ntk-smoke', is 'Completed', while the others are 'Failed'. A 'Create' button is visible in the top right of the table area.

Name	Status of last run	Last run	Runs	Frequency	Created by	Created on
ntk-smoke	Completed	ntk-smoke-0012	1	--	?????? ????????	9/6/26, 2:12:20 AM
ntk_probe_stdlib	Failed	ntk_probe_stdlib	1	--	?????? ????????	9/6/26, 1:14:31 AM
ntk_probe_trivial	Failed	ntk_probe_trivial	1	--	?????? ????????	9/6/26, 1:13:35 AM
ntk-smoke	Failed	ntk-smoke-2312	1	--	?????? ????????	9/6/26, 1:12:45 AM
ntk-smoke	Failed	ntk-smoke-2311	1	--	?????? ????????	9/6/26, 1:11:02 AM
ntk-smoke	Failed	ntk-smoke-2309	1	--	?????? ????????	9/6/26, 1:09:30 AM
ntk-smoke	Failed	ntk-smoke-2308	1	--	?????? ????????	9/6/26, 1:08:00 AM

Three ways to run a LangGraph agent



Option	Provided	You add, or accept
Container Apps	revisions, traffic splits, private ingress	protocol, per-user sandboxes, state, tracing
LangSmith Deployment	LangGraph server, threads, persistence, tracing	on Azure: self-hosted, Enterprise licence
Foundry hosted agent	Entra identity, session sandboxes, versions, traces	no traffic splits, public endpoint

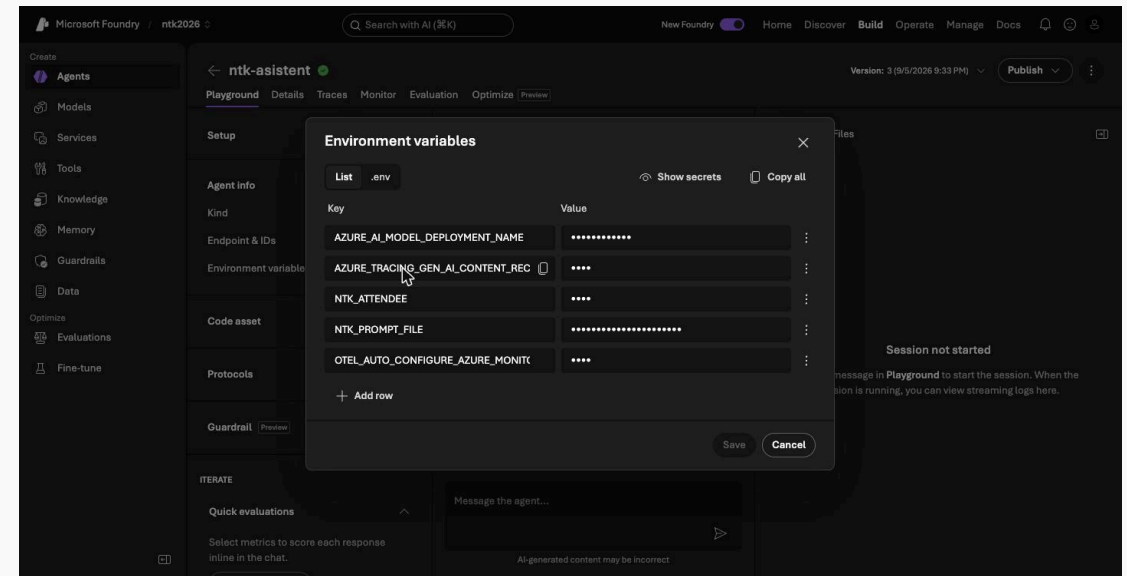
When not to use a hosted agent



- One session needs more than 2 vCPU / 4 GiB
- You need canary or A/B — traffic cannot be split
- Your agent is not written in Python or C#
- The agent's address must be in your network — not yet

Gotchas

- Changing one setting creates a new version
- There is no rollback command — it is 30 lines of script
- An empty answer instead of an error: check the model limits



Summary



- A LangGraph agent goes to production without a rewrite
- Identity, isolation, versions, traces — without writing code
- Everything shown is in the repository

What you get without writing it



Identity

The agent has its own
Entra account — not
the app's secret.



Isolation

Every conversation
gets its own sandbox.
Nothing leaks.



Versions

Every deploy is a
frozen number. Back
with one command.



Traces

Every call is
recorded: which tools,
how long.

— without a single line of your code —

Links



Repozitorij: koda, ovoj, skripte



Dokumentacija: gostovani agenti



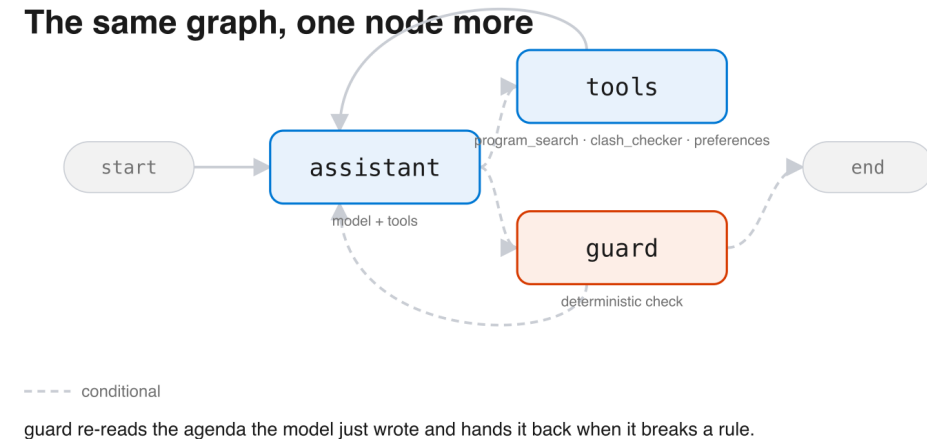
Questions

Optional: a rule in the graph



- A guard node checks the agenda after the model writes it
- A broken limit goes back to the model with the reason, at most twice
- The same prompt gives different answers; a rule in code gives one verdict
- A hosted agent runs any graph, so the rule ships as a new version

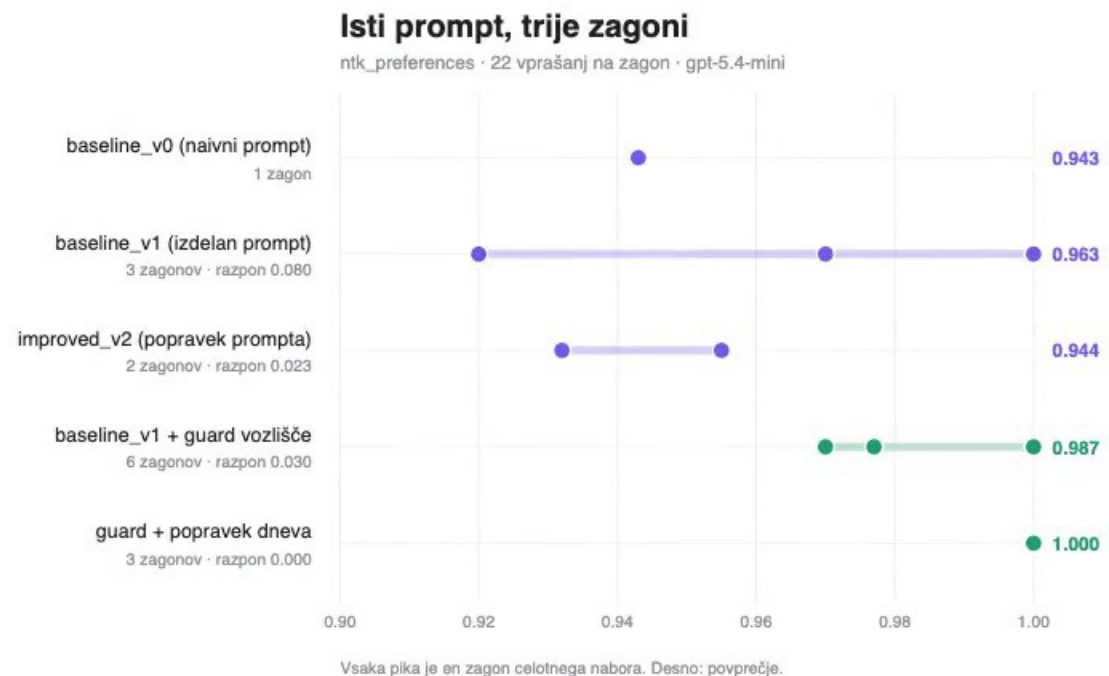
The same graph, one node more





OPTIONAL DEMO · The agent catches itself

Same prompt, three runs. The spread between runs of one prompt is wider than the difference between prompts. The chart is real, from evals/results/, drawn by uv
run eval-variance (scripts/eval_variance.py); this demo does not re-run it.



src/agent/graph.py — the body of guard_node

```
1 def guard_node(state: MessagesState) -> dict:
2     """Score the answer the model just wrote; hand the violations back, or pass.
3
4     Every decision is logged, including the passes: an approval and a guard that never ran
5     look identical from the outside, which is how the residual failures in S-08b went
6     unexplained for a day. ``attendee_source`` is the field that earns its keep.
7
8     ``RemoveMessage`` drops the rejected answer from state, and **that does not un-send it**
9     (S-08e). The Foundry hosting layer always drives the graph with ``astream`` and converts
10    node updates to Responses events as they happen, so every assistant turn is already on
11    the wire by the time the guard forms an opinion about it. The removal still keeps the
12    conversation clean for later turns; the caller sees the draft *and* the repair, which is
13    why ``contract.extract_agenda`` takes the last agenda and not the first.
14    """
15    attempt = guard.corrections_so_far(state["messages"]) + 1
16    question = guard.asked(state["messages"])[64]
17    verdict = guard.inspect_state(state)
18    problems = verdict["problems"]
```